

Ctrl+K, ctrl+D

Tạo db xong -> tải framework -> thêm String vô appsetting.json -> thêm code vào program

using Microsoft.EntityFrameworkCore;

***Framework**

Install-Package Microsoft.EntityFrameworkCore.SqlServer -Version 8.0.28

Install-Package Microsoft.EntityFrameworkCore.Tools -Version 8.0.28

Install-Package Microsoft.EntityFrameworkCore.Design -Version 8.0.28

***appsetting.json**

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
    "Server=.;Database=Fruit_Store_Management_System;Trusted_Connection=True;Trust
    ServerCertificate=True"
  }
}
```

***tạo các file cs/appdbcontext**

Scaffold-DbContext

"Server=.;Database=Fruit_Store_Management_System;Trusted_Connection=True;TrustServerCertificate=True" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models -Context

ApplicationDbContext -DataAnnotations

Scaffold-DbContext

"Server=.;Database=TEN_DATABASE;Trusted_Connection=True;TrustServerCertificate=True" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models -Context
ApplicationDbContext -DataAnnotations -Force

***program.cs**

builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection"));

***ApplicationDbContext - xóa cái này**

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) #warning To protect potentially sensitive information in your connection string, you should move it out of source code. You can avoid scaffolding the connection string by using the Name= syntax to read it from configuration - see <https://go.microsoft.com/fwlink/?linkid=2131148>. For more guidance on storing connection strings, see <https://go.microsoft.com/fwlink/?LinkId=723263>.

=>

optionsBuilder.UseSqlServer("Server=.;Database=Fruit_Store_Management_System;Trusted_Connection=True;TrustServerCertificate=True");

Connect

History Browse

Recent Connections

- (local) - <default> (sa)
- (localdb)\mssqllocaldb, <default> (LAPTOP-U1QKNGKE\LENOVO)

Connection Properties Connection String

Server Name

Authentication: SQL Server Authentication

User Name: sa

Password: ••••••

☒ Remember Password

Database Name: <default>

Encrypt: Mandatory

☒ Trust Server Certificate

Advanced...

Custom Properties

Name: (local)

Color: <default>

Custom...

Reset Connect Cancel

```

create database Fruit_Store_Management_System;
go
USE Fruit_Store_Management_System;
GO
create table Users(
    Id int identity(1,1) primary key,
    FullName nvarchar(200) not null,
    Email nvarchar(255) not null unique
        CHECK (Email LIKE '%@%._%'),
    PasswordHash nvarchar(max) not null,
    Role nvarchar(20) not null
        check (Role = 'Admin' or Role = 'User'),
    CreatedAt datetime default getdate(),)
create table Categories(
    Id int identity(1,1) primary key,
    CategoriesName nvarchar(50) not null unique ,
    Descriptions nvarchar(255),
    CreatedAt datetime default getdate(),)
create table Fruits(
    Id int identity(1,1) primary key,
    FruitsName nvarchar(50) not null unique ,
    Price decimal(10,2) not null check(Price >0),
    StockQuantity float default 0,
    CategoryId int not null,
    foreign key(CategoryId) references Categories(Id),
    CreatedAt datetime default getdate(),)
create table Orders(
    Id int identity(1,1) primary key,

```

```

        UserId int not null,
        foreign key(UserId) references Users(Id),
        OrderDate datetime not null ,
        TotalAmount decimal(10,2) not null,
        Status nvarchar(20) not null check (Status in('Pending', 'Completed',
'Cancelled')),
        CreatedAt datetime default getdate(),)
create table OrderItems(
    Id int identity(1,1) primary key,
    OrderId int not null,
    foreign key(OrderId) references Orders(Id) ,
    FruitId int not null,
    foreign key(FruitId) references Fruits(Id) ,
    Quantity int not null check(Quantity >0),
    UnitPrice decimal(10,2),
    CreatedAt datetime default getdate(),)

```

```

/*=====
DATABASE
=====*/

```

```

CREATE DATABASE FruitStore;          -- Tạo database mới tên FruitStore

USE FruitStore;                      -- Chọn database FruitStore để làm việc

DROP DATABASE FruitStore;            -- Xóa database FruitStore

```

```

/*=====
TABLE
=====*/

```

```

CREATE TABLE Users (                -- Tạo bảng Users
    UserID INT IDENTITY(1,1) PRIMARY KEY,    -- Khóa chính, tự tăng
    FullName NVARCHAR(100) NOT NULL,         -- Không được NULL
    Email VARCHAR(100) UNIQUE
        CHECK (Email LIKE '%_@_%._%'),      -- Không được trùng
    Phone VARCHAR(20),                      -- Số điện thoại
    Role NVARCHAR(20) DEFAULT 'User',        -- Mặc định là User
    Age INT CHECK (Age >= 0)                 -- Tuổi phải >= 0
);

```

```

ALTER TABLE Users
ADD Address NVARCHAR(200);              -- Thêm cột Address

```

```

ALTER TABLE Users
ALTER COLUMN Address NVARCHAR(500); -- Sửa kiểu dữ liệu cột Address

```

```

ALTER TABLE Users
DROP COLUMN Address;                   -- Xóa cột Address

```

```

DROP TABLE Users;                    -- Xóa bảng Users

```

```

/*=====
INSERT (CREATE trong CRUD)
=====*/

```

```
INSERT INTO Users
(FullName, Email, Phone, Role, Age)
VALUES
('Nguyen Van A','a@gmail.com','0909123456','Admin',20);
-- Thêm 1 dòng dữ liệu
```

```
INSERT INTO Users
(FullName, Email, Phone, Role, Age)
VALUES
('Tran Van B','b@gmail.com','0909111111','User',18),
('Le Van C','c@gmail.com','0909222222','User',22);
-- Thêm nhiều dòng dữ liệu
```

```
/*=====
SELECT (READ trong CRUD)
=====*/
```

```
SELECT *
FROM Users;
-- Lấy tất cả cột, tất cả dữ liệu
```

```
SELECT FullName, Email
FROM Users;
-- Chỉ lấy FullName và Email
```

```
SELECT DISTINCT Role
FROM Users;
-- Lấy Role không bị trùng
```

```
SELECT TOP 5 *
FROM Users;
-- Lấy 5 dòng đầu tiên
```

```
/*=====
WHERE
=====*/
```

```
SELECT *
FROM Users
WHERE UserID = 1;
-- UserID bằng 1
```

```
SELECT *
FROM Users
WHERE UserID <> 1;
-- UserID khác 1
```

```
SELECT *
FROM Users
WHERE Age > 18;
-- Tuổi lớn hơn 18
```

```
SELECT *
FROM Users
```

```
WHERE Age < 18;  
-- Tuổi nhỏ hơn 18
```

```
SELECT *  
FROM Users  
WHERE Age >= 18;  
-- Tuổi lớn hơn hoặc bằng 18
```

```
SELECT *  
FROM Users  
WHERE Age <= 18;  
-- Tuổi nhỏ hơn hoặc bằng 18
```

```
SELECT *  
FROM Users  
WHERE Age BETWEEN 18 AND 30;  
-- Tuổi từ 18 đến 30
```

```
SELECT *  
FROM Users  
WHERE UserID IN (1,2,3);  
-- UserID thuộc danh sách
```

```
SELECT *  
FROM Users  
WHERE FullName LIKE 'N%';  
-- Tên bắt đầu bằng N
```

```
SELECT *  
FROM Users  
WHERE FullName LIKE '%A';  
-- Tên kết thúc bằng A
```

```
SELECT *  
FROM Users  
WHERE FullName LIKE '%Van%';  
-- Tên chứa chữ Van
```

```
SELECT *  
FROM Users  
WHERE Phone IS NULL;  
-- Phone đang NULL
```

```
SELECT *  
FROM Users  
WHERE Phone IS NOT NULL;  
-- Phone có dữ liệu
```

```
/*=====
```

TOÁN TỬ LOGIC

```
=====*/
```

```
SELECT *  
FROM Users  
WHERE Age > 18  
AND Role = 'User';
```

-- Cả 2 điều kiện đều đúng

```
SELECT *  
FROM Users  
WHERE Role = 'Admin'  
OR Role = 'User';
```

-- Một trong hai điều kiện đúng

```
SELECT *  
FROM Users  
WHERE NOT Role = 'Admin';  
-- Không phải Admin
```

```
/*=====
```

```
ORDER BY
```

```
=====*/
```

```
SELECT *  
FROM Users  
ORDER BY FullName ASC;  
-- Sắp xếp tăng dần A-Z
```

```
SELECT *  
FROM Users  
ORDER BY FullName DESC;  
-- Sắp xếp giảm dần Z-A
```

```
SELECT *  
FROM Users  
ORDER BY Role ASC, FullName DESC;  
-- Sắp theo Role rồi FullName
```

```
/*=====
```

```
UPDATE (UPDATE trong CRUD)
```

```
=====*/
```

```
UPDATE Users  
SET Role = 'Admin'  
WHERE UserID = 2;  
-- Đổi Role của UserID=2 thành Admin
```

```
/*=====
```

```
DELETE (DELETE trong CRUD)
```

```
=====*/
```

```
DELETE FROM Users  
WHERE UserID = 2;  
-- Xóa UserID=2
```

```
DELETE FROM Users;  
-- Xóa toàn bộ dữ liệu trong bảng
```

```
TRUNCATE TABLE Users;  
-- Xóa toàn bộ dữ liệu nhanh hơn DELETE
```

```
/*=====
HÀM THỐNG KÊ
=====*/
```

```
SELECT COUNT(*)
FROM Users;
-- Đếm số dòng
```

```
SELECT SUM(Age)
FROM Users;
-- Tính tổng Age
```

```
SELECT AVG(Age)
FROM Users;
-- Tính trung bình Age
```

```
SELECT MAX(Age)
FROM Users;
-- Tuổi lớn nhất
```

```
SELECT MIN(Age)
FROM Users;
-- Tuổi nhỏ nhất
```

```
/*=====
GROUP BY
=====*/
```

```
SELECT Role, COUNT(*)
FROM Users
GROUP BY Role;
-- Nhóm theo Role
```

```
/*=====
HAVING
=====*/
```

```
SELECT Role, COUNT(*)
FROM Users
GROUP BY Role
HAVING COUNT(*) > 2;
-- Chỉ lấy nhóm có trên 2 dòng
```

```
/*=====
FOREIGN KEY
=====*/
```

```
CREATE TABLE Orders (
    OrderID INT IDENTITY(1,1) PRIMARY KEY, -- Khóa chính
    UserID INT NOT NULL,                    -- Khóa ngoại
    OrderDate DATETIME DEFAULT GETDATE(),  -- Ngày tạo đơn
)
```

```

        FOREIGN KEY (UserID)
        REFERENCES Users(UserID)           -- Liên kết Users
        CategoryId int, foreign key(CategoryId) references Categories(Id),

    );

```

```

/*=====
JOIN
=====*/

```

```

SELECT *
FROM Users U
INNER JOIN Orders O
ON U.UserID = O.UserID;
-- Chỉ lấy dữ liệu khớp nhau

```

```

SELECT *
FROM Users U
LEFT JOIN Orders O
ON U.UserID = O.UserID;
-- Lấy toàn bộ Users

```

```

SELECT *
FROM Users U
RIGHT JOIN Orders O
ON U.UserID = O.UserID;
-- Lấy toàn bộ Orders

```

```

SELECT *
FROM Users U
FULL JOIN Orders O
ON U.UserID = O.UserID;
-- Lấy toàn bộ cả hai bảng

```

```

/*=====
ALIAS
=====*/

```

```

SELECT FullName AS Name
FROM Users;
-- Đổi tên cột tạm thời thành Name

```

```

/*=====
SUBQUERY
=====*/

```

```

SELECT *
FROM Users
WHERE UserID IN (
    SELECT UserID
    FROM Orders
);
-- Truy vấn lồng nhau

```



```
/*=====
UNION
=====*/
```

```
SELECT FullName
FROM Users
```

```
UNION
```

```
SELECT FullName
FROM Customers;
```

```
-- Gộp kết quả, loại bỏ dòng trùng
```

```
SELECT FullName
FROM Users
```

```
UNION ALL
```

```
SELECT FullName
FROM Customers;
```

```
-- Gộp kết quả, giữ lại dòng trùng
```

```
/*=====
VIEW
=====*/
```

```
CREATE VIEW UserView AS
SELECT UserID, FullName
FROM Users;
-- Tạo View
```

```
SELECT *
FROM UserView;
-- Xem dữ liệu View
```

```
DROP VIEW UserView;
-- Xóa View
```

```
/*=====
INDEX
=====*/
```

```
CREATE INDEX idx_FullName
ON Users(FullName);
-- Tăng tốc tìm kiếm theo FullName
```

```
/*=====
VARIABLE
=====*/
```

```
DECLARE @Name NVARCHAR(100);
-- Khai báo biến
```

```
SET @Name = 'Nguyen Van A';  
-- Gán giá trị
```

```
PRINT @Name;  
-- In giá trị biến
```

```
/*=====
```

```
IF ELSE
```

```
=====*/
```

```
IF EXISTS (  
    SELECT *  
    FROM Users  
    WHERE UserID = 1  
)
```

```
BEGIN  
    PRINT 'Found';
```

```
END
```

```
ELSE
```

```
BEGIN  
    PRINT 'Not Found';
```

```
END
```

```
-- Kiểm tra điều kiện
```

```
/*=====
```

```
CASE
```

```
=====*/
```

```
SELECT FullName,  
    CASE  
        WHEN Age >= 18 THEN 'Adult'  
        ELSE 'Child'  
    END AS Status
```

```
FROM Users;
```

```
-- Giống switch-case
```

```
/*=====
```

```
STORED PROCEDURE
```

```
=====*/
```

```
CREATE PROCEDURE GetUsers
```

```
AS
```

```
BEGIN
```

```
    SELECT *
```

```
    FROM Users;
```

```
END;
```

```
-- Tạo Procedure
```

```
EXEC GetUsers;
```

```
-- Chạy Procedure
```

```
/*=====
```

```
TRANSACTION
```

```

=====*/

BEGIN TRANSACTION;
-- Bắt đầu transaction

UPDATE Users
SET Age = Age + 1
WHERE UserID = 1;

COMMIT;
-- Lưu thay đổi

ROLLBACK;
-- Hủy thay đổi


/*=====
DATE FUNCTIONS
=====*/

SELECT GETDATE();
-- Lấy ngày giờ hiện tại

SELECT YEAR(GETDATE());
-- Lấy năm

SELECT MONTH(GETDATE());
-- Lấy tháng

SELECT DAY(GETDATE());
-- Lấy ngày

SELECT DATEADD(DAY,7,GETDATE());
-- Cộng thêm 7 ngày

SELECT DATEDIFF(DAY,'2025-01-01','2025-12-31');
-- Tính số ngày giữa 2 mốc


/*=====
STRING FUNCTIONS
=====*/

SELECT UPPER('hello');
-- HELLO

SELECT LOWER('HELLO');
-- hello

SELECT LEN('Hello');
-- Độ dài chuỗi

SELECT SUBSTRING('Hello World',1,5);
-- Lấy chuỗi con

SELECT LTRIM('    Hello');
-- Xóa khoảng trắng bên trái

```

```
SELECT RTRIM('Hello   ');  
-- Xóa khoảng trắng bên phải
```